

Physics-Informed Neural Networks

Marko Ristić
Los Alamos National Laboratory

Petnica Summer School on
Scientific Machine Learning

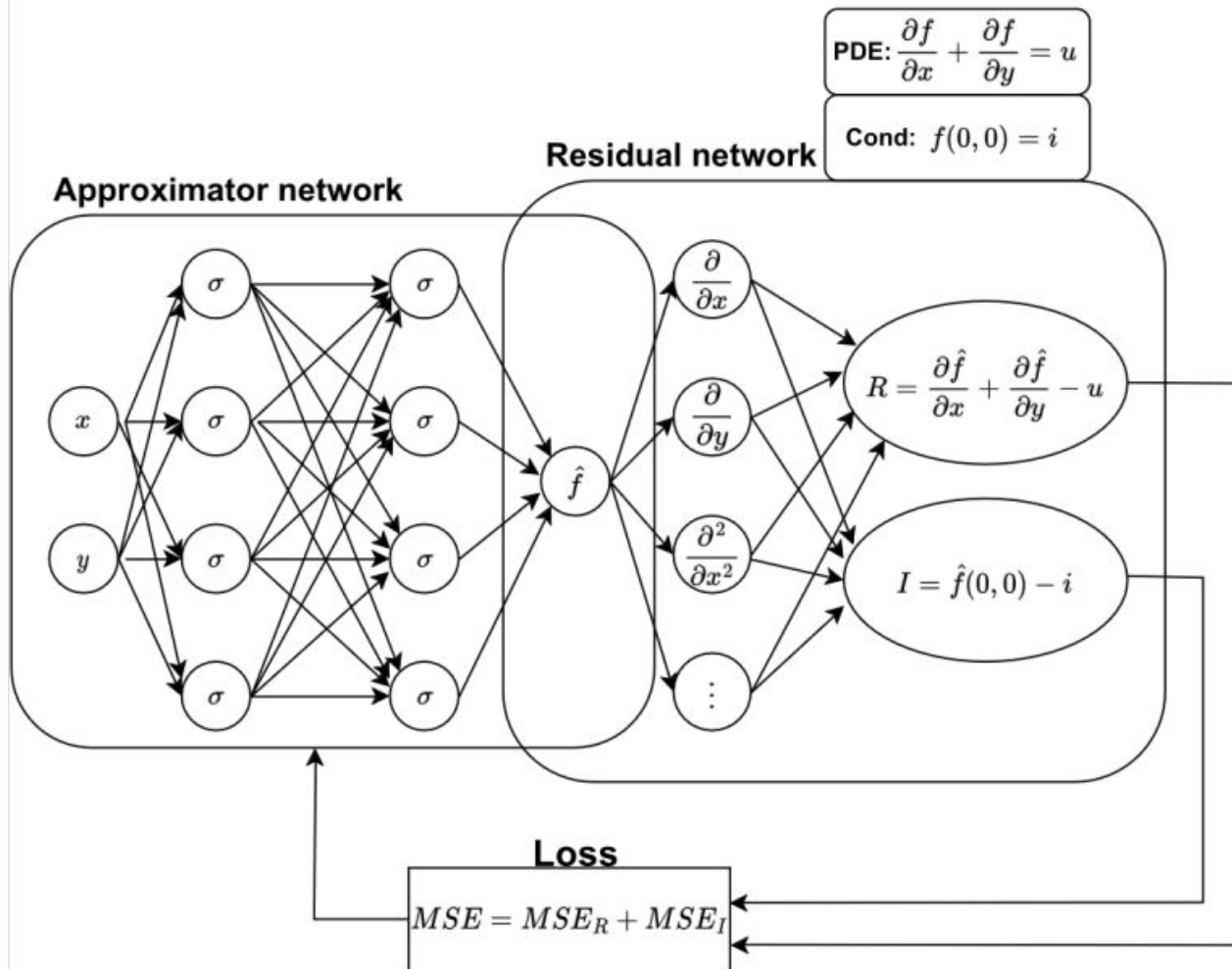
About Me

- Marko Ristić
- Postdoc at Los Alamos National Laboratory (soon at UC San Diego)
- Research interests: astrophysics, kilonova modeling, radiative transfer, surrogate models, Bayesian inference, PINNs
- I do lots of research and academic mentoring (PhD apps, CV review, etc.); reach out!
 - <https://mristic.space>
 - markoris@ucsd.edu
- I have 5 cats!



Physics-Informed Neural Networks (PINNs)

Neural networks which directly embed governing differential equations into their loss definition to find solutions which are enforced by the laws of nature.



When are PINNs useful?

**Full physics description,
well-defined geometry**

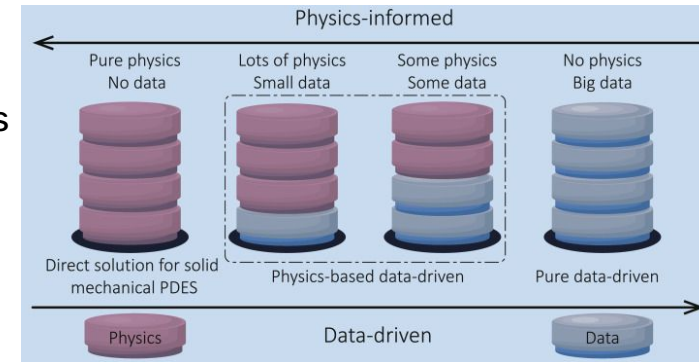
Use traditional numerical methods (finite element, Euler's, Runge-Kutta, etc.) to find an exact solution for a given set of assumptions

**No governing equations,
unclear input domain**

Use supervised machine learning (assuming some semi-understood map between inputs and outputs exists) or unsupervised machine learning to try and describe experimental data

**Partial physics,
unknown parameters,
sparse experiment data**

Use PINNs!



Forward Model vs. Inverse Problem

- Traditional Forward Modeling
 - Produces a single solution for a fixed set of assumptions, usually propagating theory through some numerical method to generate a final state
- PINNs
 - Generates a solution for the forward problem through constraints imposed by the differential equation **while also enabling inverse problems by learning a field solution!**

Computational Tradeoffs

Traditional Methods

- Tailored to a specific problem formulation
- Once formulated, can efficiently simulate specific formulations of given parameter combinations
- Changing parameters/inputs requires running new simulations (typically requires **many CPU-hrs**)

PINNs

- Learns a solution field across a parameter domain
- Requires long training time (typically much longer than a single simulation **using GPU-hrs**)
- Allows easy comparison to data by evaluating across the solution field (**much cheaper than simulation**)

Ok, you have decided that PINNs are suitable for your problem. How do they differ from standard supervised learning?

Inputs and Outputs

Supervised Learning

Inputs: Samples drawn from the assumed data distribution (physical quantities, model parameters, etc.)

Outputs: A label, class, or prediction

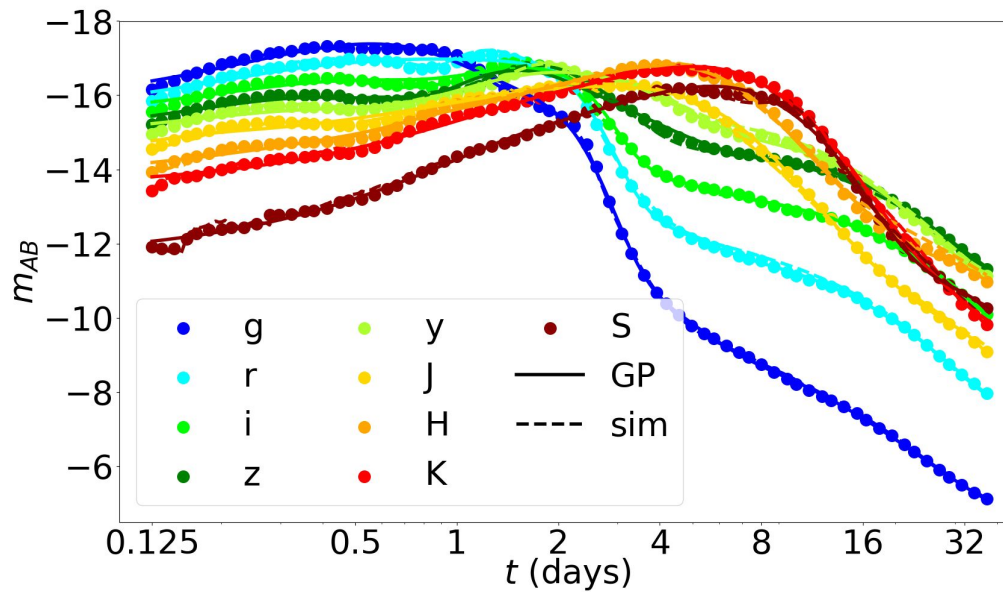
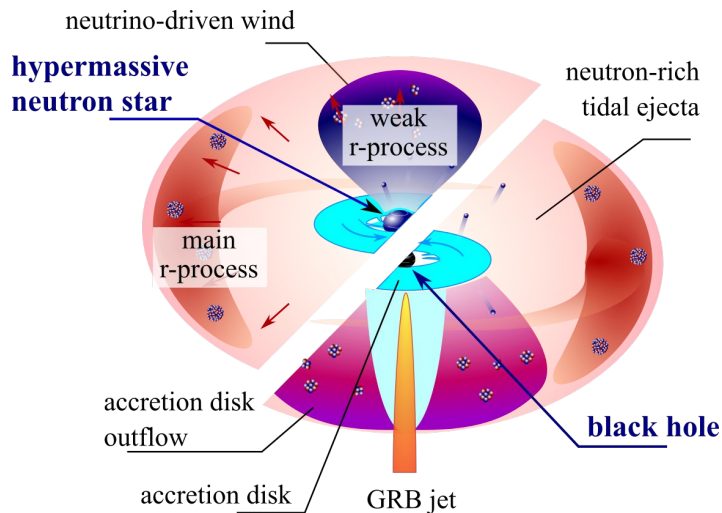
PINN

Inputs: Coordinates in the physical domains of the independent variables and equation parameters

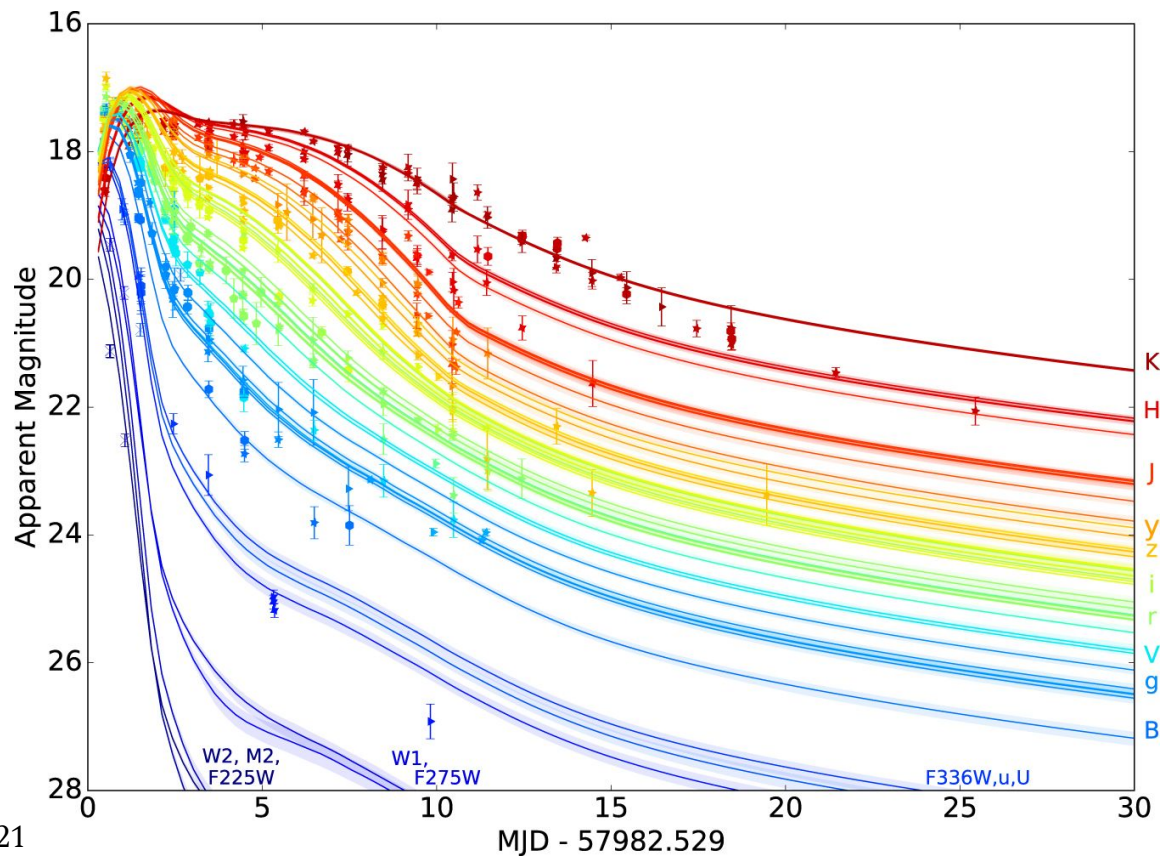
Outputs: The physical quantity predicted by the function at the given coordinate(s)

Supervised Learning

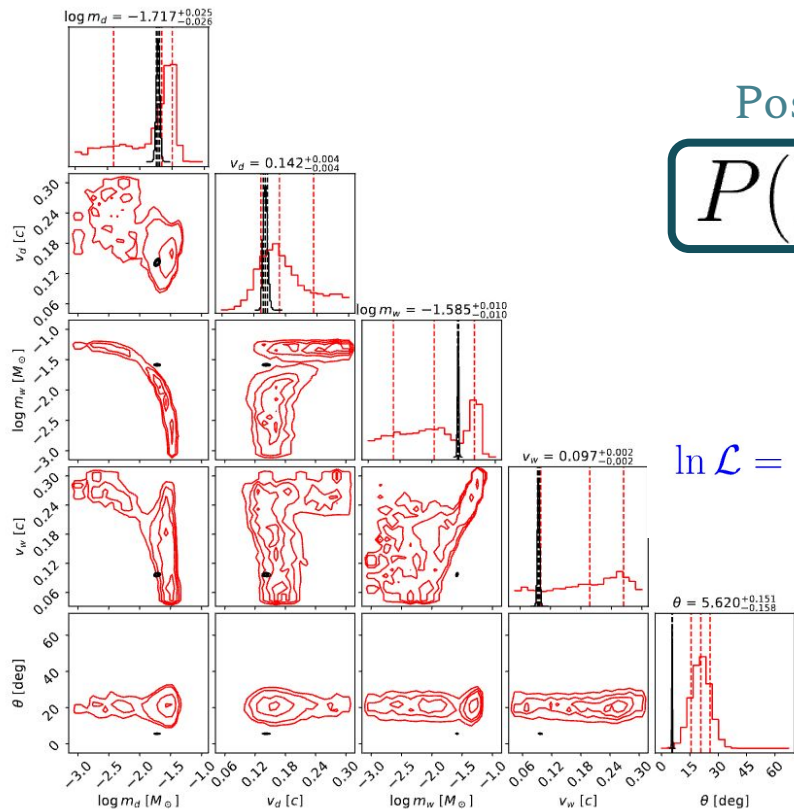
- Masses: m_d , m_w
- Velocities: v_d , v_w



We have some data!!



The Inverse Problem



$$\text{Posterior } P(A|B) = \frac{\text{Prior } P(A) \times \text{Likelihood } P(B|A)}{\text{Marginal } P(B)}$$

$$\ln \mathcal{L} = -\frac{1}{2} \left(\sum_i \left[\frac{(\hat{y}_i - d_i)^2}{\sigma_i^2 + \sigma_{\text{sys},i}^2} \right] + \ln \left[(2\pi)^N \sum_i (\sigma_i^2 + \sigma_{\text{sys},i}^2) \right] \right)$$

What is being learned?

Supervised Learning

A map from features to labels, valid across an assumed data distribution

PINN

A single continuous field $f(x, t, \theta)$ which is the solution to the differential equation

Reusability

Supervised Learning

Trained once and then reapplied to unlimited new inputs **with the caveat that these new inputs are well represented by the assumed data distribution**

PINN

Works for an assumed problem setup (**boundary condition, geometry, or parametrization**). As soon as any of these change, PINN must be **retrained**

Inductive Bias (“Guidance” for Solution Space)

Supervised Learning

The architecture choices encode the bias:

Convolution

Spatial locality + translational and rotational invariance

Attention

Importance of components in sequence

Recurrence

Complexity of the persistent behavior

PINN

The loss fully guides the allowable solution space through physically motivated constraints, **prioritizing solutions which are representative of physically realistic states**

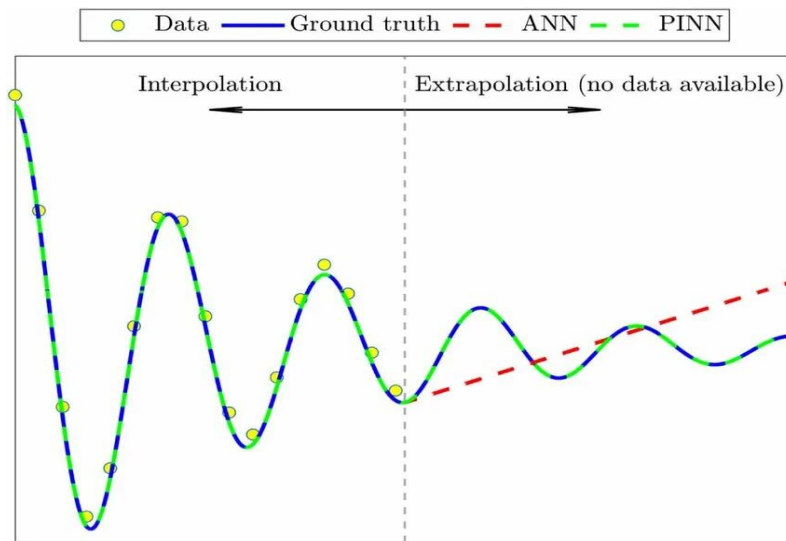
Behavior Away from Data (Extrapolation)

Supervised Learning

Extremely unreliable, famously bad at generalizing outside of the training data

PINN

Governed by the physics embedded in the equation wherever it is enforced;



Loss Terms

Supervised Learning

Usually one: some scalar representation of the network's performance (e.g. mean squared error)

PINN

Multiple terms:

- initial condition loss
- + boundary value loss
- + physics/equation loss
- + (optional) data loss
- = total PINN loss

Each of these has a **weight** which modifies its **contribution** to the total PINN loss

Residual Meaning

Supervised Learning

Prediction error, general representation of the mismatch between the mapped function and the ground truth labels/predictions

PINN

Equation violation: directly probes how well the network is satisfying the physical constraint

A Zero Residual: Useful?

Supervised Learning

Objectively yes: a small (near-zero) residual almost always implies that the neural networks successfully fits the data (modulo bad normalizations, scale discrepancies, etc.)

PINN

Not necessarily: there exist trivial solutions (e.g. $f(x) = 0$ or $f(x) = \text{const.}$) which satisfy many PDEs exactly but are not interesting or informative

Gradient Calculations

Supervised Learning

Gradient calculation only for the **network parameters (weights and biases)** which is back-propagated to reduce the loss

PINN

Gradient calculation for both the **network parameters** and whichever **physical parameters** have gradient dependence on the independent variable (see Miron's presentation re: directed acyclic graphs)

Activation Function Restrictions

Supervised Learning

Rarely any, problem-dependent but otherwise generally all activation functions are feasible unless obvious failure points

PINN

Explicitly demand **second-order (or higher) differentiable** activation functions; the first derivative (or higher) comes from the equation, and backpropagation differentiates the function with respect to the network parameters

Methods and Motivations from Numerical Solutions of PDEs

Nondimensionalization

- Removing dimensionality from parameters and (usually) normalizing simultaneously

Take the dimensional heat/diffusion equation:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}$$

1. Substitute $x = Lx^*$, $t = Tt^*$, and $u = Uu^*$:

$$\frac{U}{T} \frac{\partial u^*}{\partial t^*} = D \frac{U}{L^2} \frac{\partial^2 u^*}{\partial (x^*)^2}$$

2. Divide through by $\frac{U}{T}$ to isolate the time derivative:

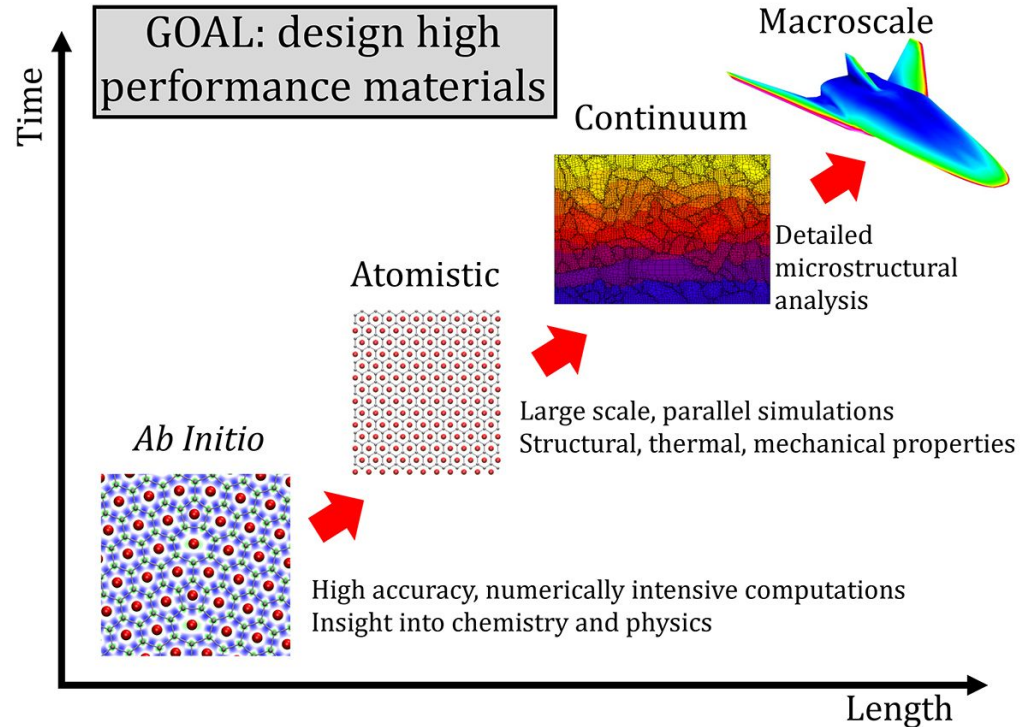
$$\frac{\partial u^*}{\partial t^*} = \left(\frac{DT}{L^2} \right) \frac{\partial^2 u^*}{\partial (x^*)^2}$$

3. Choose the characteristic time scale $T = \frac{L^2}{D}$ to make the prefactor equal to 1, yielding the fully dimensionless PDE:

$$\frac{\partial u^*}{\partial t^*} = \frac{\partial^2 u^*}{\partial (x^*)^2}$$

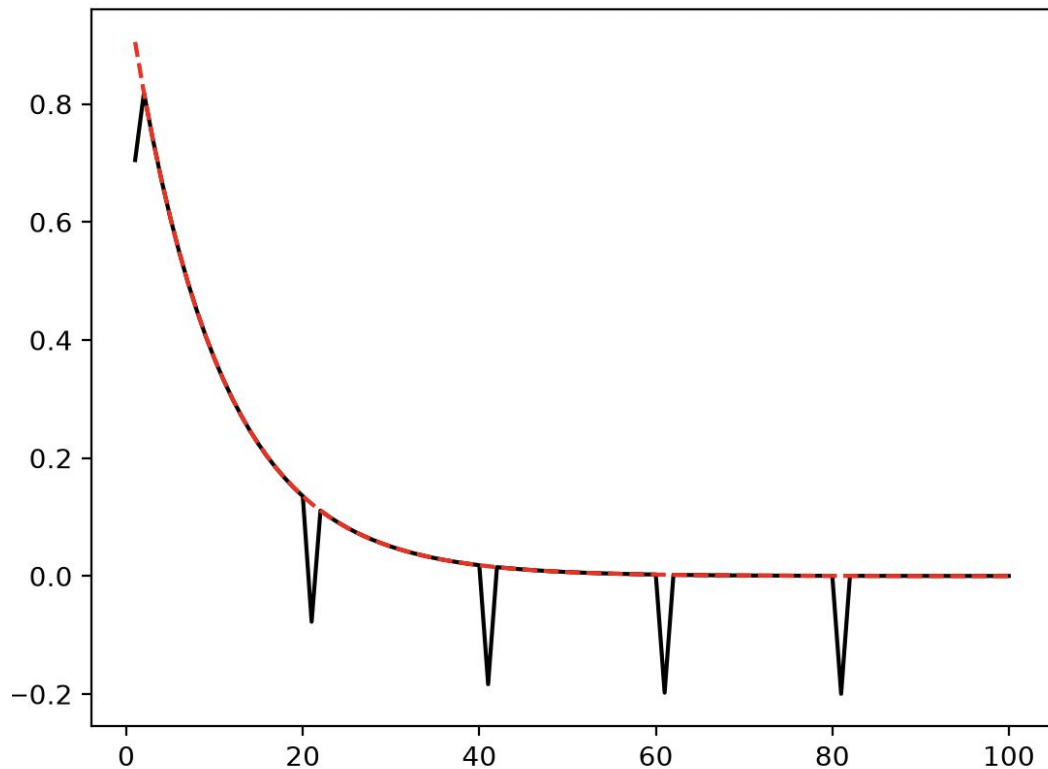
Stiffness: PDE Solvers vs. PINNs

- In traditional PDE solving methods, a stiff PDE is one with substantially different spatial or temporal scales which exist simultaneously
 - Forces intractably small timesteps to maintain numerical stability
- For PINNs, stiffness manifests as substantially different gradient magnitudes in a multi-term loss



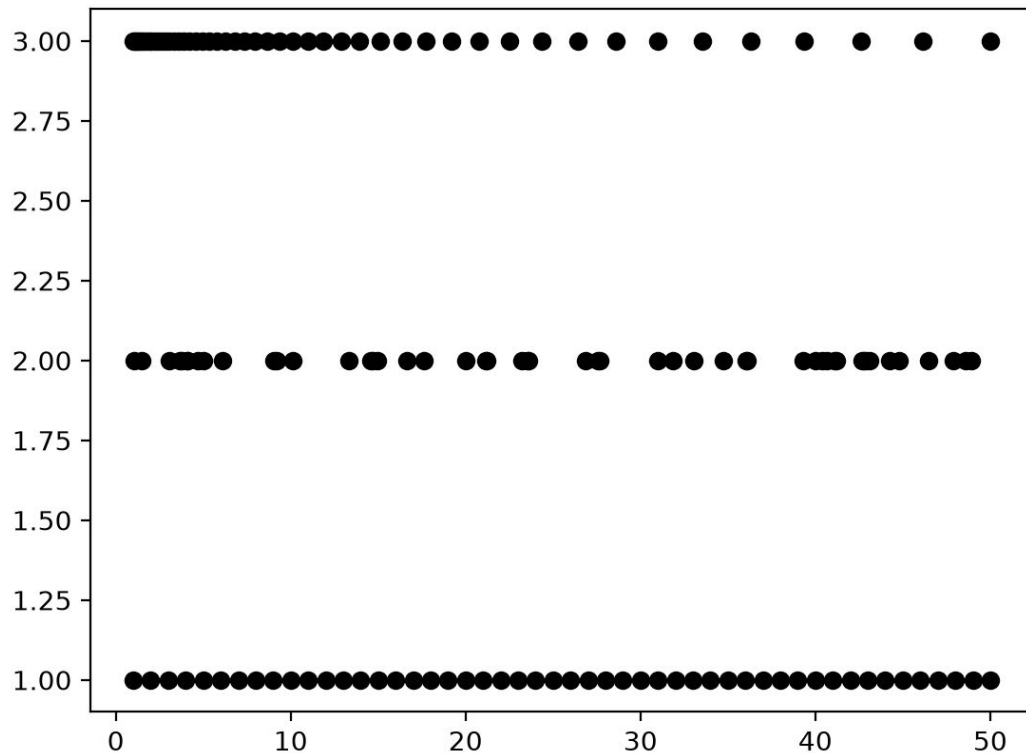
PINN Stiffness and Annealing

What is one way to actively implement annealing during training?



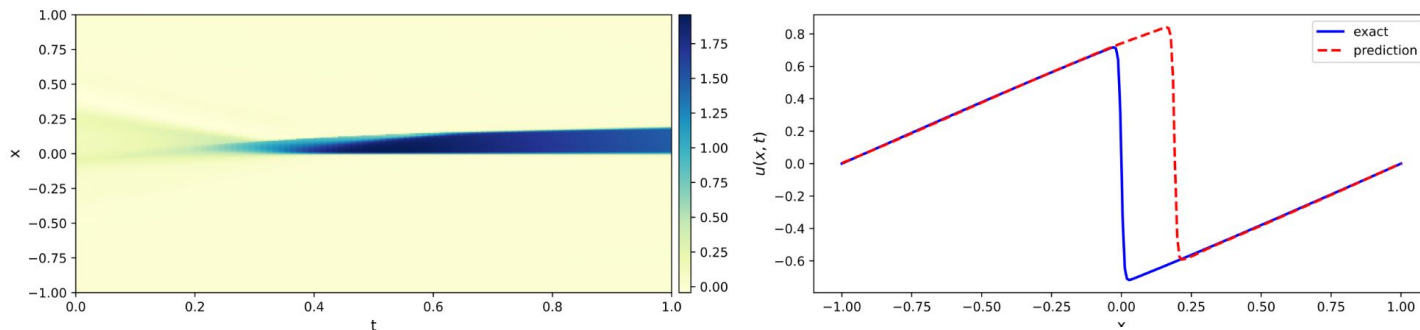
Collocation Point Sampling

When to use which?



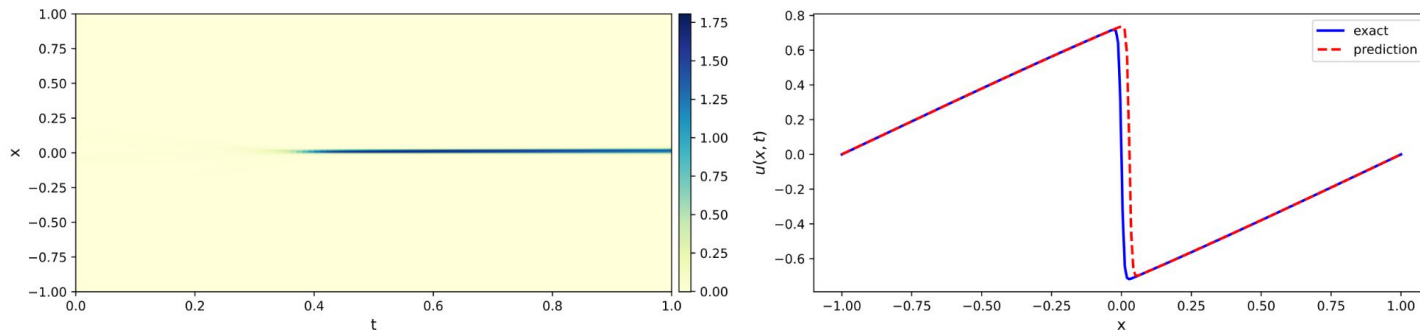
Residual-Based Sampling

Numerical results obtained by *Uniform*

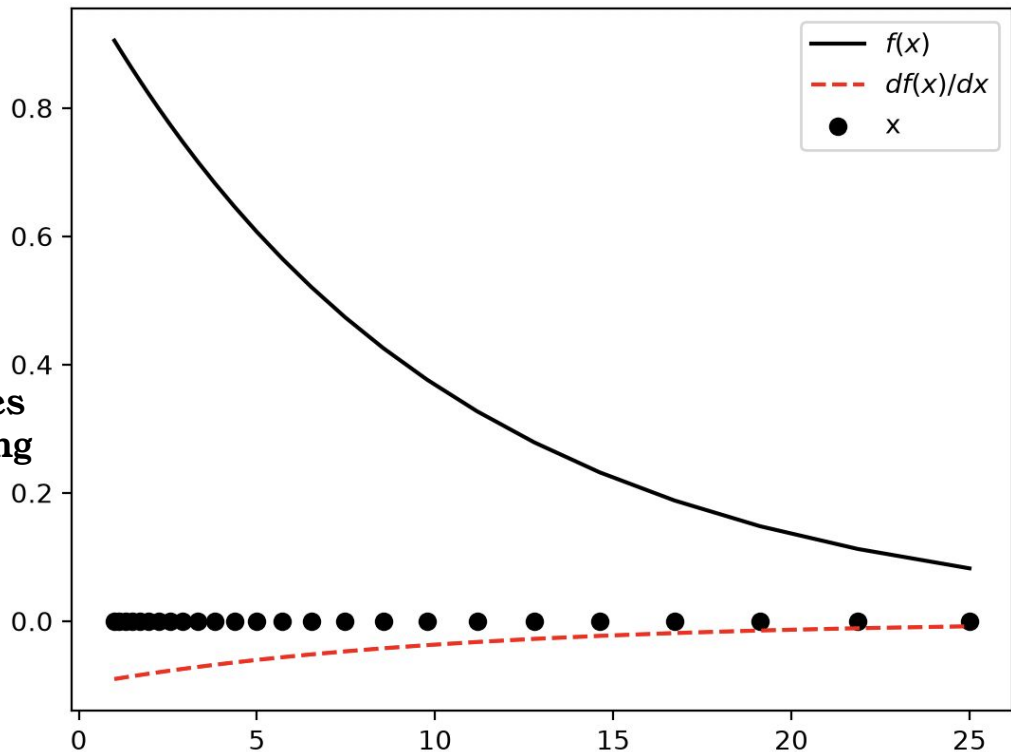


Failure points?

Numerical results obtained by Residual-based sampling

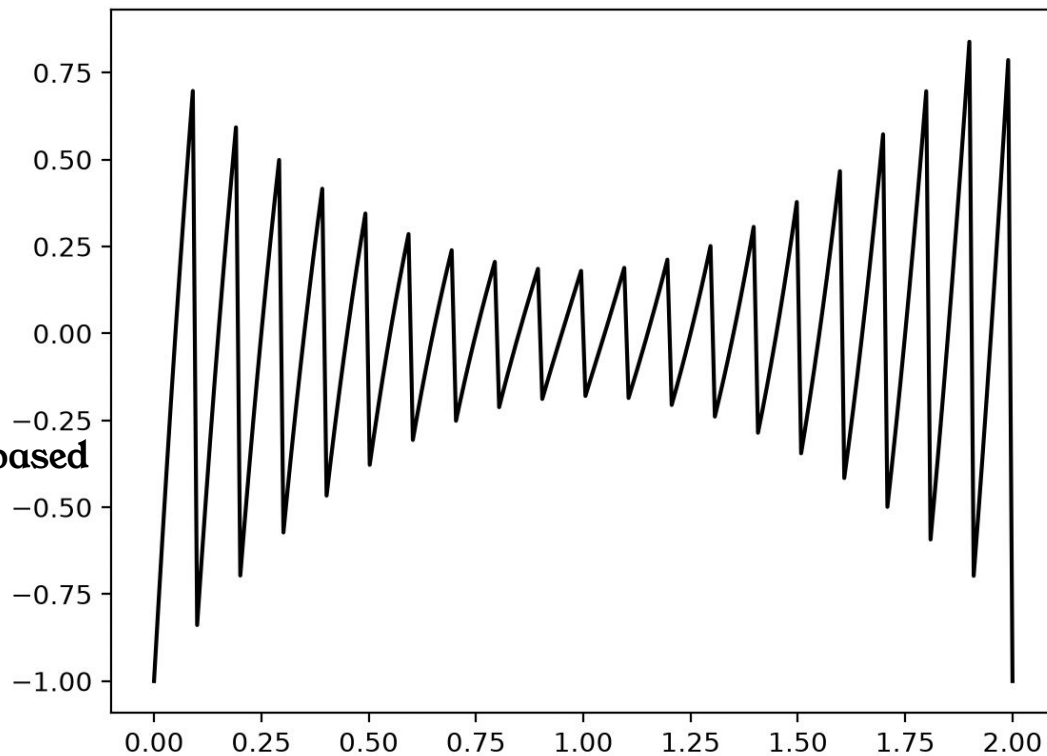


Feature-based Sampling



What assumption does feature based sampling explicitly make?

Parameter-based Weighting



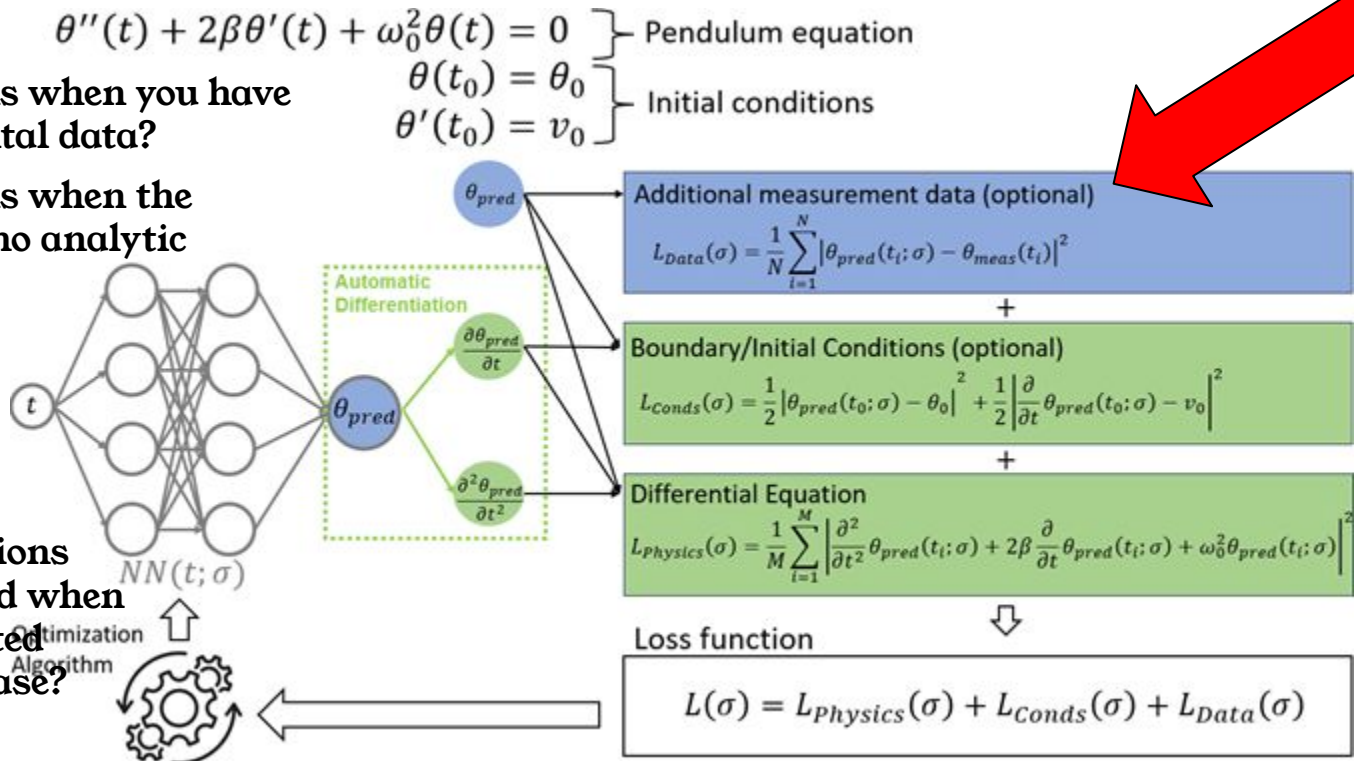
What assumption
(might) parameter based
weighting make?

Data-driven Simulation

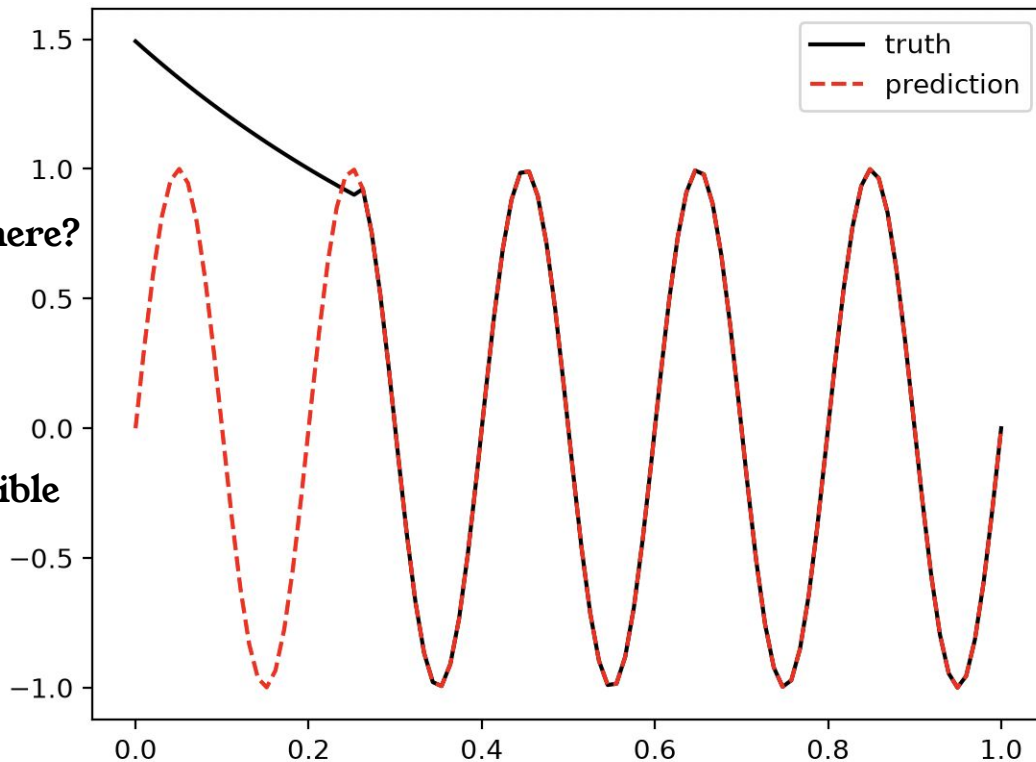
What happens when you have no experimental data?

What happens when the problem has no analytic solution?

What limitations are introduced when using simulated data in this case?



Loss Weighting



What is happening here?

How do you fix this?

**What are some possible
consequences when
applying the fix?**